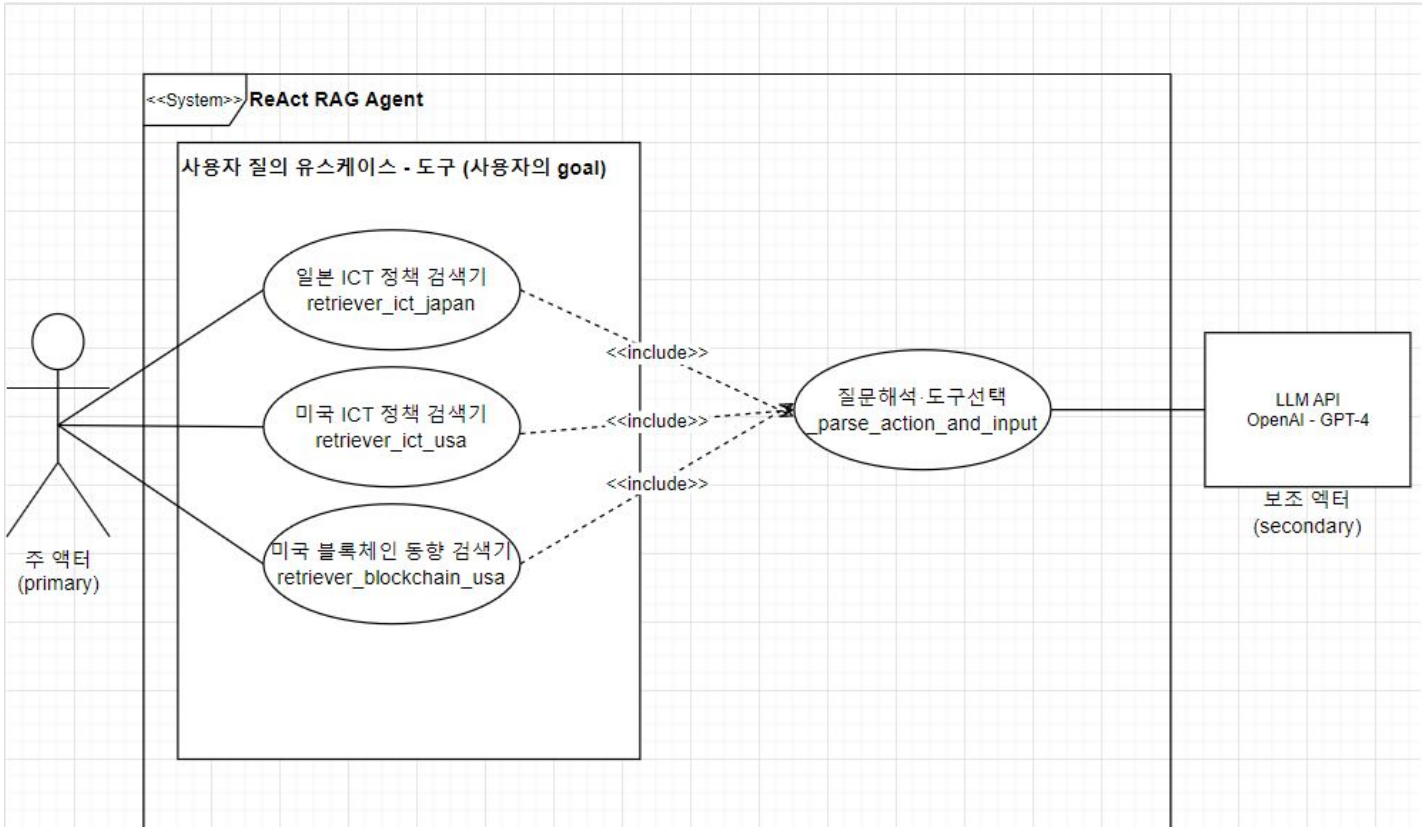
ch03
ReAct / Function Calling Agent UML 작성

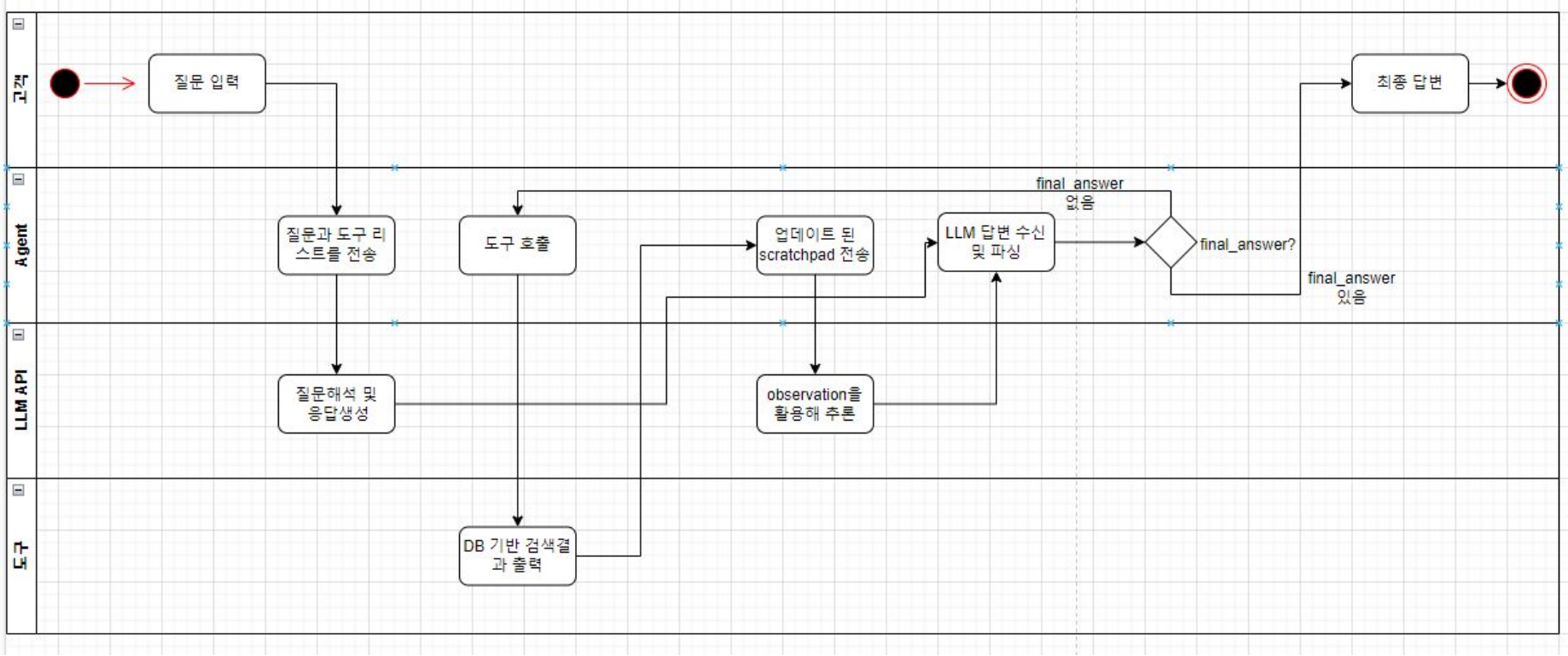
2026-10-21 노동민

ReAct Agent

1. Use-Case Diagram



2. Activity Diagram



ReAct Agent

질문입력
- user_input, scratchpad 은 _render_prompt() 함수로
- 도구리스트는 _format_tools_for_promt() 함수로 문자열변환 후 _render_promt()로

-> LLM은 파이썬 코드를 실행할 권한이 없기 때문에 객체정보를 알지 못함.
LLM이 읽을 수 있는 구조로 렌더링한 redered를 이용해 llm.invoke() 호출

질문해석 및 응답생성 (1차 LLM 응답)
- 제공된 정보를 바탕으로 어떤 도구를 사용하고 어떤 Input을 사용해야할지 스스로 추론
- 추론 결과를 프롬프트엔지니어링 한 형식으로 출력 -> agent 가 코드를 실행해 정규식으로 정보 추출

ReAct Agent

추출된 정보는 tool 과 action_input 이며 이 값을 바탕으로 검사 실행

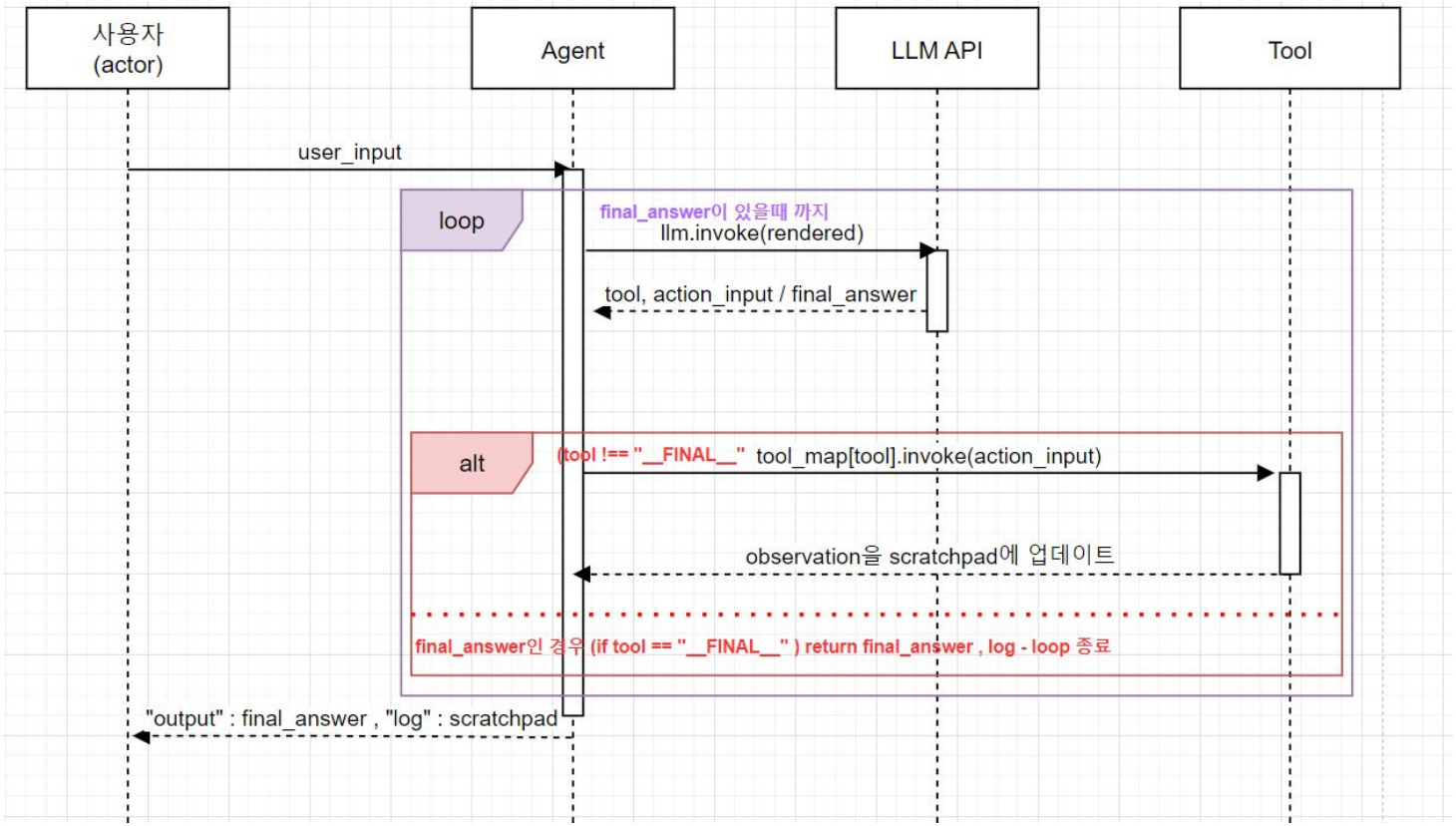
값을 추출해내는 함수인 _parse_action_and_input 은 LLM이 출력한 text 에서 Final answer 가 존재하면 tool 값에 “__FINAL__”을 저장

if 문을 활용해 최종답변이 존재하면 output을 리턴하고 종료

tool 에 도구의 이름이 정상적으로 들어 왔다면 observation(관측)값을 얻기 위해 tool_map[tool].invoke(action_input) 실행

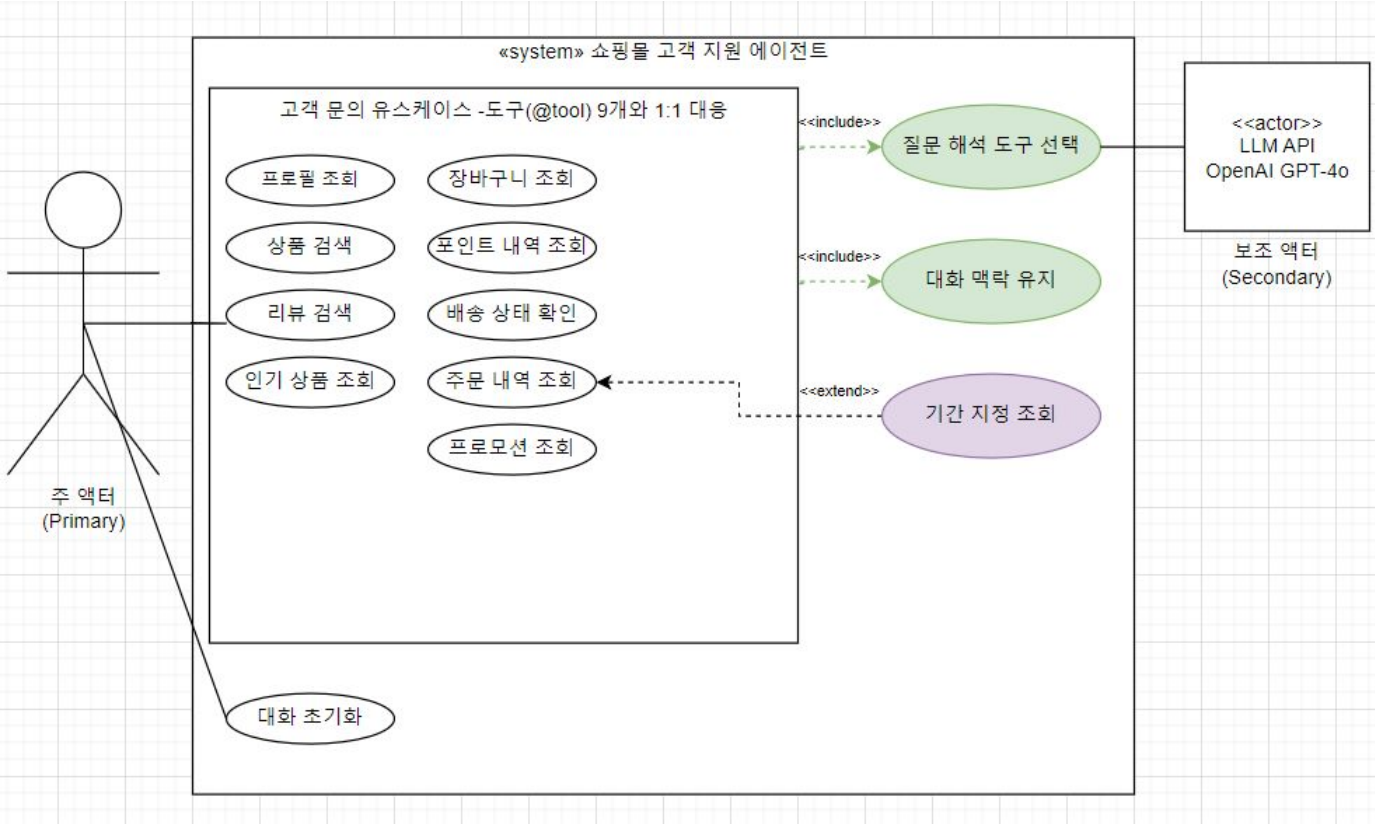
+ 실행된 결과는 _observation_to_text()를 통해 LLM이 읽을 수 있는 text 형식으로 변환 (metadata,page_content 등의 요소를 꺼내 문자열로 변환 및 크기 조절)

3. Sequence Diagram

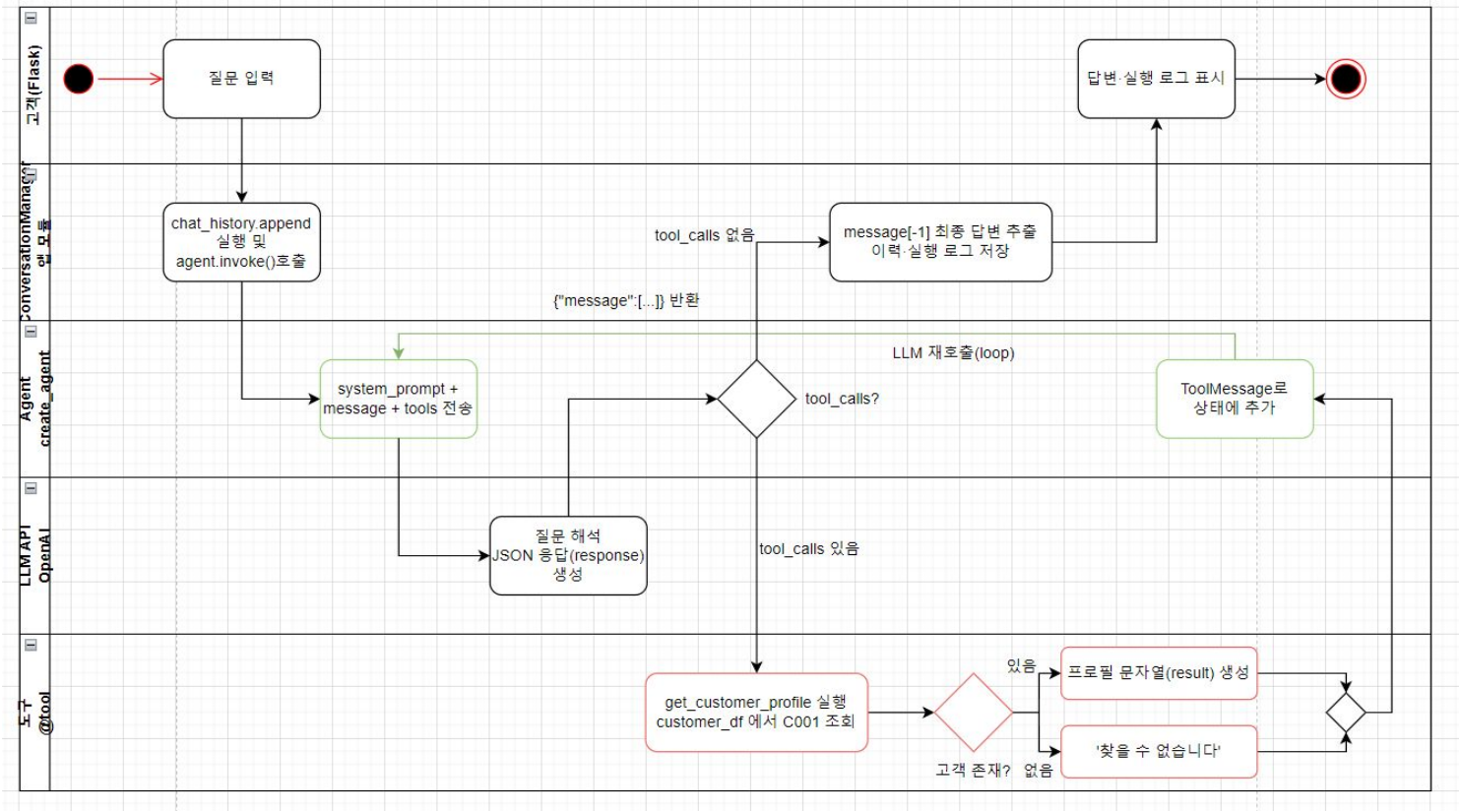


Function Calling Agent

1. Use-Case Diagram



2. Activity Diagram



Function Calling Agent - ReAct와 비교

@tool 은 개발자가 작성한 함수를 agent가 활용할 수 있는 도구로 만들어줌.
(JSON 형식의 스키마로 변환해주는 기능을 제공)

ConversationManager 는 왜 Class로 정의 되어 있는가?
-> 사용자 세션별로 개별적인 히스토리와 질의를 처리하기 위함

사용자 질의 입력시 chat_history에 append 되는 구조의 이유?
-> 첫 질문 이후에 LLM이 과거 문맥을 파악할 수 있게 해줌.

log 의 messages 는
사용자 질의 - HumanMessage
LLM의 답변 - AIMessage
도구의 결과 - ToolMessage 로 이루어져 있음

Function Calling Agent - ReAct와 비교

JSON 형식의 출력(Function Call)은 개발자가 직접 문자를 정규식으로 파싱하지 않아도, 구조화된 출력으로 값을 검증, 저장하기 용이 (if hasattr(msg, ‘tool calls’) and msg.tool_calls:

3. Sequence Diagram

